

Capstone Project Report

Automating an E-Commerce Web Application

Using Selenium WebDriver with Python

A Comprehensive End-to-End Testing Framework

Student Name: Angshul Sana

Enrollment Number: 12023002001023

Course/Institution: Btech in CSE at Institution of Engineering and Management Kolkata

Submission Date: 28/Sept/2026

Submitted in partial fulfillment of the requirements for the degree

2. Certificate

This is to certify that the project report entitled "**Automating an E-Commerce Web Application Using Selenium WebDriver with Python**" submitted by **Angshul Sana** (Enrollment No: 12023002001023), in partial fulfillment of the requirements for the degree of Btech in CSE at Institution of Engineering and Management Kolkata, is an authentic record of the student's own work carried out under supervision.

The matter embodied in this report has not been submitted by the student for the award of any other degree or diploma.

Supervisor Signature

Date:

HOD Signature

Date:

3. Declaration

I, **Angshul Sana**, hereby declare that the capstone project entitled "**Automating an E-Commerce Web Application Using Selenium WebDriver with Python**" submitted to the Institution of Engineering and Management Kolkata, is a record of original work done by me.

I further declare that this project report has not been previously formed the basis for the award of any degree, diploma, fellowship, or similar title of recognition to the best of my knowledge and belief.

Signature of Student

Angshul Sana

Date: 28/Sept/2026

4. Acknowledgement

I would like to express my profound gratitude to the faculty and administration of the Institution of Engineering and Management Kolkata for providing the necessary facilities and guidance throughout this capstone project.

Special thanks to my project guide and mentors for their invaluable support, constructive feedback, and technical assistance, which were instrumental in the successful completion of this automation framework. I am also grateful to my peers and family for their continuous encouragement.

5. Abstract

In the modern software development lifecycle, ensuring the reliability and functional accuracy of web applications is paramount. This project presents a robust, automated end-to-end testing framework developed for an e-commerce platform (TutorialsNinja Demo).

Utilizing Python, Selenium WebDriver, and the Pytest framework, the project automates critical user journeys including secure login, product search, cart manipulation, and data validation. The framework is architected using the Page Object Model (POM) design pattern to ensure high maintainability and modularity. External test data management is facilitated via JSON and Excel (openpyxl), while environment variables securely handle credentials. The execution pipeline integrates dynamic wait mechanisms, alert handling, screenshot capture for visual evidence, and comprehensive HTML report generation. The resulting framework provides a scalable foundation for continuous integration and automated quality assurance in e-commerce environments.

Table of Contents

1. Cover Page	1
2. Certificate	2
3. Declaration	2
4. Acknowledgement	3
5. Abstract	3
6. Introduction	5
7. Problem Statement and Business Scenario	5
8. Objectives	5
9. Scope of the Project	5
10. Technology Stack	6
11. System Requirements	6
12. Framework Architecture	7
13. Project Folder Structure	7
14. Detailed Module Description	8
15. Test Data Management	9
16. Test Scenario and Test Steps	9
17. Test Case Table	10
18. Execution Workflow	11
19. Screenshot Evidence	12
20. Test Execution Report	12
21. Results and Validation	13
22. Challenges and Solutions	13
23. Limitations	13
24. Future Enhancements	14
25. Conclusion	14
26. References	14

6. Introduction

In the highly competitive e-commerce sector, a seamless user experience is critical for retaining customers and ensuring business success. Manual testing of complex workflows—such as searching for products, updating carts, and calculating totals—is time-consuming and prone to human error. This project demonstrates the automation of a customer purchase journey using Python and Selenium WebDriver on the public *TutorialsNinja* e-commerce demo website (<https://tutorialsninja.com/demo/>).

7. Problem Statement and Business Scenario

Business Scenario: A customer wants to securely log in, search for a specific product, add it to their shopping cart, adjust the quantity, and verify that the billing totals are correctly calculated before proceeding to checkout.

Problem Statement: To validate this workflow efficiently across multiple iterations, a reliable automation framework is required. The framework must interact with web elements, handle dynamic synchronization (loading times), manage test data externally, secure sensitive credentials, and produce human-readable execution reports without placing irreversible test orders.

8. Objectives

- To design and implement an end-to-end web test automation framework using Python and Selenium.
- To apply the Page Object Model (POM) design pattern for scalable and maintainable code.
- To integrate Pytest for robust test execution and HTML report generation.
- To demonstrate dynamic test-data handling using JSON and Excel integrations.
- To handle synchronization issues and JavaScript alerts seamlessly.
- To capture and manage visual evidence (screenshots) of test executions.

9. Scope of the Project

The scope encompasses the automation of the core pre-checkout workflow on the *TutorialsNinja* demo application. It includes browser initialization, user authentication, inventory search, cart manipulation (adding items and changing quantities), and mathematical validation of cart totals. The automation intentionally stops before order submission to prevent irreversible transactions. Cross-browser support is provided for Google Chrome (default) and Firefox.

10. Technology Stack

The project utilizes a modern Python-based automation stack, leveraging industry-standard libraries to ensure reliability and ease of maintenance.

Category	Technology / Tool	Purpose
Programming Language	Python 3.x	Core scripting and framework logic.
Automation Tool	Selenium WebDriver	Browser automation and DOM interaction.
Test Framework	Pytest	Test execution, assertions, and discovery.
Reporting	pytest-html (4.1.1)	Generating self-contained HTML execution reports.
Data Management	openpyxl (3.1.5), JSON	Reading and writing test data externally.
Environment Config	python-dotenv (1.0.1)	Securing credentials and runtime configurations.
Browsers Supported	Google Chrome, Mozilla Firefox	Cross-browser test execution (managed via Selenium Manager).
IDE & Version Control	VS Code, Git, GitHub	Code development, versioning, and repository management.

11. System Requirements

The following hardware and software prerequisites are necessary to execute the automation framework:

Requirement Type	Specification
Operating System	Windows 10/11, macOS, or Linux
Processor	Intel Core i3 / AMD Ryzen 3 or higher
RAM	Minimum 4 GB (8 GB recommended)
Disk Space	500 MB for framework, Python environment, and browsers
Software Installed	Python 3.8+, Google Chrome (latest version)

12. Framework Architecture

The automation framework is built on the **Page Object Model (POM)** design pattern. POM enhances test maintenance and reduces code duplication. Under this architecture, every web page in the application is represented by a dedicated Python class. These classes encapsulate the Selenium web element locators and the actions that can be performed on them. The test scripts (Pytest files) invoke these page methods rather than interacting with the WebDriver directly.

This layered approach separates test intent (assertions and workflow) from implementation mechanics (locators and clicking). If the UI of the TutorialsNinja application changes, only the corresponding Page Object class needs updating, leaving the test scripts untouched.

13. Project Folder Structure

The framework is organized into a modular directory structure to separate data, utilities, page objects, and test execution files.

```
project root/
├─ pytest.ini
├─ myenv/
└─ Capstone Project/
    ├─ pages/
    │   ├─ base_page.py
    │   ├─ home_page.py
    │   ├─ login_page.py
    │   ├─ search_page.py
    │   └─ cart_page.py
    ├─ tests/
    │   ├─ conftest.py
    │   ├─ test_data_sources.py
    │   └─ test_purchase_workflow.py
    ├─ utils/
    │   ├─ browser.py
    │   ├─ config.py
    │   ├─ data_reader.py
    │   └─ helpers.py
    ├─ data/
    │   ├─ test_data.json
    │   └─ test_data.xlsx
    ├─ reports/
    │   └─ execution_report.html
    ├─ screenshots/
    ├─ .env
    ├─ .env.example
    ├─ requirements.txt
    └─ README.md
```

14. Detailed Module Description

Module / File	Responsibility
test_purchase_workflow.py	The main end-to-end test runner. Orchestrates the workflow from login to cart verification and executes Pytest assertions.
conftest.py	Contains Pytest fixtures. Handles setup/teardown, initializes the browser, manages screenshots, and configures the HTML report.
pages/base_page.py	Contains reusable Selenium wrappers (explicit waits, clicking, sending keys, handling alerts). All other page classes inherit from it.
pages/home_page.py	Navigates to the login screen and executes the search bar functionality.
pages/login_page.py	Inputs secured credentials and validates successful authentication.
pages/search_page.py	Selects the desired item from search results and initiates the 'Add to Cart' action.
pages/cart_page.py	Updates quantities and extracts tabular data (product name, unit price, total price) for assertions.
utils/config.py	Parses the .env file using python-dotenv to provide secure access to credentials and timeouts.

14.1 Configuration and Security

Sensitive information, such as the demonstration email and password, are removed from the source code. They are stored in a local `.env` file which is excluded from version control via `.gitignore`. The framework dynamically loads variables like `BASE_URL`, `BROWSER`, `HEADLESS`, and `ACTION_DELAY` at runtime.

14.2 Synchronization and Demonstration

To ensure stability against network latency, the framework strictly avoids arbitrary `time.sleep()` commands, utilizing **Explicit Waits** (`WebDriverWait` with `Expected Conditions`) instead. However, for academic and presentation purposes, an `ACTION_DELAY` is configured to simulate human typing character-by-character, making the automated execution visually observable.

15. Test Data Management Using JSON and Excel

Hardcoding test data limits framework reusability. This project implements a dual data-source strategy using JSON and Excel.

The primary configuration seed is stored in `test_data.json`. Upon execution, the framework verifies the existence of an Excel workbook. If absent, it automatically generates `test_data.xlsx` containing a "PurchaseData" sheet populated with the JSON seed data. The primary test workflow (`test_purchase_workflow.py`) then reads the product ("iPhone") and quantity ("2") dynamically from the Excel file using the `openpyxl` library.

Parameter	JSON Seed Value	Excel Extracted Value
Base URL	<code>https://tutorialsninja.com/demo/</code>	-
Target Product	iPhone	iPhone
Target Quantity	2	2
Currency Marker	\$	\$

16. Test Scenario and Test Steps

The primary scenario ensures a user can log in, find a specific device, and accurately manipulate cart metrics. The steps executed by the script are:

- Initialization:** Launch the chosen browser, maximize the window, and navigate to the Base URL. Clear stale screenshots from previous runs.
- Authentication:** Navigate to the account login page. Enter the email and password retrieved from the `.env` file. Click Login.
- Search:** Enter the product name ("iPhone") retrieved from the Excel file into the global search bar and submit.
- Add to Cart:** Locate the first matching product in the search results grid and click the "Add to Cart" button.
- Alert Handling:** Safely poll the DOM for any unexpected JavaScript alerts (accepting them if present, ignoring if absent).
- Cart Modification:** Open the shopping cart page. Locate the quantity input field, clear it, and input the new quantity ("2"). Update the cart.
- Validation:** Extract the product name, unit price, and line total from the cart table. Assert that the mathematical product of unit price and quantity matches the displayed total.
- Evidence Collection:** Capture screenshots at key milestones and generate the Pytest HTML report.

17. Test Case Table

The following table outlines the formal test case executed by the automated script, detailing the expected and actual outcomes.

Test Case ID	Objective	Steps Executed	Expected Result	Actual Result	Status
TC_E2E_001	Verify End-to-End Purchase Workflow (Pre-Checkout)	1. Open browser. 2. Navigate to URL. 3. Enter valid credentials & login. 4. Search for "iPhone". 5. Click 'Add to Cart'. 6. Handle JS alert if present. 7. Navigate to Cart View. 8. Update quantity to 2. 9. Extract prices & calculate.	User successfully logs in. Product is added. Cart updates quantity to 2. Displayed total price matches (Unit Price $\times$ 2). No unhandled exceptions occur.	Login successful. Product added to cart. Quantity updated to 2. Mathematical assertion passed successfully.	PASS

17.1 Advantages of the Framework

Feature	Advantage Provided
Page Object Model	UI locators are centralized. High reusability and low maintenance overhead.
Data-Driven Setup	Test execution is decoupled from hardcoded data; easily scales to new products.
Robust Synchronization	Explicit waits eliminate flaky tests caused by page load delays.
Automated Evidence	Screenshots and HTML reports provide immediate visual feedback on failures.
Security	Environment variables ensure passwords are never exposed in reports or Git.

18. Execution Workflow

The diagram below illustrates the sequential flow of actions executed by the automation framework during a standard test run. This linear progression ensures all dependent preconditions are satisfied before evaluating the final cart mathematics. By using a compact sequential pipeline, the framework ensures robust, repeatable test states.



19. Screenshot Evidence

Visual evidence is automatically captured by the framework at critical test junctures. This is facilitated by a custom Pytest fixture that utilizes Selenium's `save_screenshot()` method. Screenshots are named dynamically with timestamps and saved in the `screenshots/` directory.

- **Login Verification:** Confirms the dashboard is accessible.
- **Cart Addition:** Verifies the UI success message when the product is added.
- **Final Cart State:** Captures the updated cart totals for manual audit against the automated assertions.
- **Failure Screenshots:** If any assertion fails or an element is not found, an automatic snapshot is taken to aid debugging.

20. Test Execution Report

Reporting is handled by the `pytest-html` plugin. Upon completion of the test suite, a self-contained HTML file is generated at `reports/execution_report.html`.

The report includes:

- **Environment Details:** Python version, OS platform, and installed plugins.
- **Summary Metrics:** Total tests passed, failed, skipped, and execution duration.
- **Test Case Status:** Individual breakdown of `test_purchase_workflow.py` results.
- **Tracebacks:** Detailed logs for any failed assertions.

(Refer to the attached project submission assets and demonstration video for the physical HTML report file and execution recordings).

21. Results and Validation

The execution of the automated framework successfully validated the target business scenario. The test dynamically read the "iPhone" data from the generated Excel file, authenticated securely, and added the item to the cart.

The core logic validation—updating the quantity to 2 and verifying the mathematical accuracy of the cart—passed successfully. The assertions confirmed that the expected product name was present and that $(\text{Unit Price} \times 2) = \text{Total Line Price}$. Screenshots and the HTML report were successfully generated.

22. Challenges and Solutions

Challenge Encountered	Implemented Solution
Unpredictable page loading times causing <code>ElementNotInteractableException</code> .	Implemented <code>WebDriverWait</code> with <code>expected_conditions</code> to wait until elements were strictly visible or clickable.
Unexpected JavaScript alerts blocking execution flow.	Added an alert handling utility that polls for <code>alert_is_present()</code> , accepts it if found, and safely ignores it if absent.
Demonstrating the automation visually during presentations.	Created a custom <code>type_text_slowly</code> wrapper method that delays keystrokes based on the <code>ACTION_DELAY</code> environment variable.
Securing personal/demo credentials in GitHub.	Integrated <code>python-dotenv</code> to read variables from a git-ignored <code>.env</code> file, providing a dummy <code>.env.example</code> for structure.

23. Limitations

- The automation interacts with a public demo application (TutorialsNinja). If the site undergoes maintenance or changes its HTML structure, the Page Object locators will require updating.
- The framework intentionally halts before the final checkout step, meaning payment gateway integrations and final order confirmations are not tested in this iteration.
- Performance and load testing are outside the scope of this Selenium-based functional testing framework.

24. Future Enhancements

While the current framework is fully functional for the defined scope, several enhancements can be implemented to scale the project for enterprise-level deployment:

- **Data Parameterization:** Utilize Pytest's `@pytest.mark.parametrize` to run the same workflow across multiple rows of Excel data (e.g., testing different products like MacBooks or cameras simultaneously).
- **CI/CD Integration:** Configure GitHub Actions or Jenkins pipelines to trigger test executions automatically upon code commits or on a nightly schedule.
- **Parallel Execution:** Integrate `pytest-xdist` to run tests concurrently across different browsers, reducing overall suite execution time.
- **Advanced Reporting:** Replace `pytest-html` with Allure Reports for richer analytics, trend graphs, and historical test data tracking.
- **Checkout Flow:** Implement the full checkout process using a mocked payment gateway in a controlled, private staging environment.

25. Conclusion

This capstone project successfully demonstrates the design, development, and execution of a robust web automation framework using Python, Selenium WebDriver, and Pytest. By employing the Page Object Model, the framework achieves a high degree of modularity and maintainability.

The successful integration of external data sources (Excel/JSON) and secure configuration management (dotenv) reflects industry best practices. The automated validation of cart mathematics, combined with automatic screenshot generation and HTML reporting, proves the framework's capability to effectively replace manual regression testing for critical e-commerce workflows.

26. References

- Selenium WebDriver Documentation: <https://www.selenium.dev/documentation/>
- Pytest Framework Documentation: <https://docs.pytest.org/>
- TutorialsNinja Demo Application: <https://tutorialsninja.com/demo/>
- Openpyxl Documentation: <https://openpyxl.readthedocs.io/>
- Python Dotenv Documentation: <https://pypi.org/project/python-dotenv/>
- Demonstration Video: [Google Drive Link](#)